I D X E X C H A N G E

AI Agentic Engineer Intern Handbook

OpenClaw-Based Multi-Agent AI Engineering Internship

Summer 2026 • 12 Weeks • Production-Ready AI Systems

12 Weeks	Hybrid	667K+	OpenClaw + AI
Program Duration	Format	Data Records	Tech Stack

Core Databases

`rets_property`

~228K active MLS listings | 130+ fields

`california_sold`

~439K sold transactions | 46 fields

What You Will Build

A production multi-agent AI assistant capable of real-time MLS property search, market analytics, semantic recommendations, RAG knowledge retrieval, and WhatsApp + email communication — powered by OpenClaw.

Program Overview

This internship teaches you to build modern, production-grade AI agent systems using OpenClaw as the core runtime and orchestration framework. You will work directly with two real MLS datasets totaling over 667,000 records — giving you hands-on experience with the kind of live data that powers real estate platforms serving millions of buyers and agents nationwide.

Learning Objectives

- Design and implement multi-agent AI architectures with OpenClaw
- Build natural language interfaces over structured relational databases
- Apply vector embeddings and semantic search to property data
- Create retrieval-augmented generation (RAG) pipelines
- Integrate WhatsApp and email as AI communication channels
- Engineer safe, production-ready agentic workflows with human-in-the-loop guardrails

Your Two Core Databases

rets_property — Active MLS Listings

~228,410 active California property listings. 130+ fields including full listing remarks, photo URL arrays (Cotality/Trestle CDN), agent contact info, HOA details, architectural style, parcel numbers, and rich timestamp history. The live search and discovery table.

california_sold — Sold Transactions & Comps

~439,167 sold, leased, and closed transactions across California spanning 2021–2025. 46 fields covering close price, days on market, agent names, coordinates, property type, and all key physical attributes. The historical comps and market analytics table.

Weekly Schedule at a Glance

Week	Module	Primary Focus
0	Environment Setup	OpenClaw install, MySQL import, WhatsApp config, API keys
1	Architecture Fundamentals	OpenClaw runtime, skills, sessions, tool calling, memory
2	NL Property Search	NLP query parsing over rets_property
3	Database Integration	Parameterized MySQL queries, pagination, result formatting
4	Conversational Agent	Multi-turn session memory, follow-up handling
5	Market Analytics	SQL aggregations over california_sold comps data
6	Embeddings & Vector Search	Semantic property matching with OpenAI embeddings
7	Recommendation Engine	Similarity scoring across rets_property & sold comps
8	RAG Pipeline	Document-aware agent with MLS field definitions

9	Multi-Agent Orchestration	Coordinator routing across all specialized agents
10	WhatsApp Layer	End-to-end WhatsApp AI communication interface
11	Email Agents & Safety	Automated email workflows with approval guardrails
12	Capstone Demo	Full production multi-agent assistant presentation

Data Schema Reference

Both databases live in the same MySQL schema (boxgra5_cali). You will query them individually and join them via shared fields such as ListingKey / L_ListingID, city, postal code, and coordinates. Understanding the schema deeply is fundamental to every week in this program.

rets_property — Active Listings

Column	Type	Description
id	INT PK	Auto-increment primary key (current max ~228,410)
L_ListingID	VARCHAR	MLS system listing ID — joins to california_sold.ListingKey
L_DisplayId	VARCHAR	Human-readable MLS number shown on portals
L_Address	VARCHAR	Full street address
L_City	VARCHAR	City — indexed for fast city-based queries
L_Zip	VARCHAR	Postal code — indexed
L_Class	VARCHAR	Property class: Residential, CommercialSale, Land, etc.
L_Type_	VARCHAR	Subtype: SingleFamilyResidence, Condominium, etc. — indexed
L_Keyword2	INT	Bedrooms total
LM_Dec_3	DECIMAL(4,1)	Bathrooms total (supports half-baths e.g. 2.5)
L_SystemPrice	INT	Current list price (search/display price)
LM_Int2_3	INT	Approximate finished square footage
L_Keyword1	VARCHAR	Lot size (string, often in sq ft or acres)
LMD_MP_Latitude	DECIMAL(18,15)	Geo latitude — high precision
LMD_MP_Longitude	DECIMAL(19,15)	Geo longitude — high precision
L_Status	VARCHAR	Listing status: Active, Pending, Withdrawn, etc.
L_Remarks	MEDIUMTEXT	Full listing description — FULLTEXT indexed (ft_remarks)
L_Photos	LONGTEXT	JSON array of Cotality/Trestle photo URLs
LA1_UserFirstName	VARCHAR	Listing agent first name
LA1_UserLastName	VARCHAR	Listing agent last name
ListAgentEmail	VARCHAR	Listing agent email address
ListAgentDirectPhone	VARCHAR	Listing agent direct phone
LO1_OrganizationName	VARCHAR	Listing office / brokerage name
ListingContractDate	DATE	Date listing agreement was signed
YearBuilt	INT	Year property was constructed
SubdivisionName	VARCHAR	Subdivision or community name
AssociationFee	INT	Monthly HOA fee in dollars

Column	Type	Description
AssociationAmenities	TEXT	HOA amenities: Golf, Pool, Tennis, etc.
DaysOnMarket	INT	Days on market at time of data pull
PoolPrivateYN	VARCHAR	Private pool present (True/False)
FireplaceYN	VARCHAR	Fireplace present (True/False)
ViewYN	VARCHAR	Has a notable view (True/False)
View	VARCHAR	View description: Mountains, Ocean, GolfCourse, etc.
LotSizeAcres	DECIMAL(10,4)	Lot size in acres
LotSizeSquareFeet	DECIMAL(14,2)	Lot size in square feet
PreviousListPrice	DECIMAL(12,0)	Prior list price — enables price reduction analysis
StandardStatus	VARCHAR	RESO standard status: Active, Pending, Closed
CountyOrParish	VARCHAR	County name (e.g., Riverside, Los Angeles)
ParcelNumber	VARCHAR	Assessor parcel number (APN)
Cooling	VARCHAR	Cooling system type
Heating	VARCHAR	Heating system type
ArchitecturalStyle	VARCHAR	Architectural style: Modern, Ranch, Mediterranean, etc.
PhotoCount	INT	Number of listing photos available
ModificationTimestamp	DATETIME	Last modification timestamp for incremental sync

california_sold — Sold Transactions

Column	Type	Description
ListingKey	BIGINT	Unique listing identifier — joins to rets_property.L_ListingID
ClosePrice	DOUBLE	Final sale/close price
CloseDate	VARCHAR	Date the transaction closed (YYYY-MM-DD format)
OriginalListPrice	DOUBLE	Original asking price when first listed
ListPrice	DOUBLE	List price at time of contract
DaysOnMarket	BIGINT	Days from listing to contract
PropertyType	VARCHAR	Residential, Land, ResidentialLease, CommercialSale, etc.
PropertySubType	VARCHAR	SingleFamilyResidence, Condominium, Duplex, etc.
LivingArea	DOUBLE	Finished living area in square feet
LotSizeAcres	DOUBLE	Lot size in acres
LotSizeSquareFeet	DOUBLE	Lot size in square feet
BedroomsTotal	DOUBLE	Number of bedrooms
BathroomsTotalInteger	DOUBLE	Number of bathrooms
YearBuilt	DOUBLE	Year property was built
City	VARCHAR	City of the property

Column	Type	Description
PostalCode	VARCHAR	ZIP code
Latitude	DOUBLE	Geographic latitude
Longitude	DOUBLE	Geographic longitude
UnparsedAddress	VARCHAR	Full street address
ListAgentFirstName	VARCHAR	List agent first name
ListAgentLastName	VARCHAR	List agent last name
ListAgentFullName	VARCHAR	List agent full name
BuyerAgentFirstName	VARCHAR	Buyer agent first name
BuyerAgentLastName	VARCHAR	Buyer agent last name
ListOfficeName	VARCHAR	Listing brokerage name
BuyerOfficeName	VARCHAR	Buyer brokerage name
PoolPrivateYN	VARCHAR	Private pool (True/False/empty)
ViewYN	VARCHAR	Has view (True/False/empty)
FireplaceYN	VARCHAR	Has fireplace (True/False/empty)
NewConstructionYN	VARCHAR	New construction (True/False)
GarageSpaces	DOUBLE	Number of garage spaces
AssociationFee	DOUBLE	Monthly HOA fee
SubdivisionName	VARCHAR	Subdivision / community name
HighSchoolDistrict	VARCHAR	School district name
ListingContractDate	VARCHAR	Date listing was entered (YYYY-MM-DD)
PurchaseContractDate	VARCHAR	Date offer was accepted

Key Join Pattern

To correlate active listings with sold comps: JOIN rets_property r ON CAST(r.L_ListingID AS UNSIGNED) = cs.ListingKey — or match on city + postal code for market-level analysis.

WEEK 0 1 Week

Environment Setup & Configuration

Goals

- Install OpenClaw locally and configure the development environment
- Import both MLS SQL datasets into local MySQL
- Configure WhatsApp integration via QR code linking
- Set up all required API keys securely using environment variables
- Verify end-to-end agent communication pipeline

Repository & Node Setup

```
# Clone and install OpenClaw
git clone https://github.com/openclaw/openclaw.git
cd openclaw
npm install
openclaw onboard

# Python environment
python3 -m venv venv
source venv/bin/activate
pip install pandas openai mysql-connector-python sqlalchemy scikit-learn numpy
```

MySQL Database Import

Import both datasets into the idx_exchange schema. The rets_property table includes a FULLTEXT index on L_Remarks — allow additional time for this to build on first import.

```
# Create the database
mysql -u root -p -e "CREATE DATABASE idx_exchange CHARACTER SET utf8mb4;"

# Import active listings (~228K rows, 130+ columns)
mysql -u root -p idx_exchange < rets_property.sql

# Import sold comps (~439K rows, 46 columns)
mysql -u root -p idx_exchange < california_sold.sql

# Verify row counts
mysql -u root -p idx_exchange -e "SELECT
  (SELECT COUNT(*) FROM rets_property) AS active_listings,
  (SELECT COUNT(*) FROM california_sold) AS sold_comps;"
```

Environment Variables

```
# .env — never commit this file
OPENAI_API_KEY=sk-...
```

```
MYSQL_HOST=localhost
MYSQL_USER=idx_user
MYSQL_PASSWORD=your_secure_password
MYSQL_DATABASE=idx_exchange

EMAIL_USER=your_email@gmail.com
EMAIL_PASSWORD=your_app_password
```

WhatsApp Integration

```
openclaw channels login --channel whatsapp
# Scan the QR code in WhatsApp > Linked Devices
```

Deliverable

Both tables imported and verified. Agent sends and receives a test WhatsApp message. All API keys confirmed working.

WEEK 1
1 Week

OpenClaw Architecture Fundamentals

Core Concepts

OpenClaw is a multi-agent orchestration runtime that handles skill routing, session state, channel integration, and tool execution. Understanding how these pieces interact is the foundation for every subsequent week.

Architecture Flow
User → WhatsApp → OpenClaw Runtime → Skill Selector → Tool Execution → Memory Update → Response → User

Key Components

- **Skills — modular capability units (property search, market stats, RAG, etc.)**
- Channels — communication interfaces: WhatsApp, email, web
- Sessions — per-user conversation state and memory
- Tools — typed async functions the agent can call
- Memory — short-term session state + long-term vector storage
- Orchestrator — routes queries to the correct skill/agent

Basic Tool Definition

```
export async function getCurrentTime() {
  return { currentTime: new Date().toISOString() };
}

export async function handleMessage(message: string) {
  if (message.toLowerCase().includes("time")) {
    return await getCurrentTime();
  }
  return { response: "I could not understand the request." };
}
```

Deliverable
Architecture documentation with a workflow diagram showing how user queries flow from WhatsApp through OpenClaw skills to the MLS databases.

WEEK 2
1 Week

Natural Language Property Search

Goal

Parse free-text real estate queries into structured filter objects that can be used to query rets_property. This is the natural language front-end for the database layer built in Week 3.

> **Example Query**
> "Show me 3-bedroom condos in Irvine under $1.5M with a pool."

Supported Filters (maps to rets_property columns)

User Intent	DB Column	Example Value
city	L_City	"Irvine", "Newport Beach"
max price	L_SystemPrice	1500000
min bedrooms	L_Keyword2	3
min bathrooms	LM_Dec_3	2.5
min sq ft	LM_Int2_3	1800
property type	L_Type_	"Condominium"
pool	PoolPrivateYN	"True"
view	ViewYN	"True"
max HOA	AssociationFee	500

TypeScript NLP Parser

```
export async function parsePropertyQuery(query: string) {
  const cityMatch   = query.match(/in ([A-Za-
z\s]+?)(?:\s+under|\s+with|\s+at|$)/i);
  const priceMatch  = query.match(/under \$?([\d,.]+)(k|m)?/i);
  const bedsMatch   = query.match(/(\d+)[\s-]*(bed|beds|bedroom|bedrooms)/i);
  const bathsMatch  = query.match(/(\d+(?:\.5)?)[\s-]*(bath|baths|bathroom)/i);
  const sqftMatch   = query.match(/(\d+)[\s,]*(sqft|sq ft|square feet)/i);
  const poolMatch   = /pool/i.test(query);
  const viewMatch   = /view/i.test(query);

  const typeMap: Record<string,string> = {
    condo: "Condominium", townhome: "Townhouse",
    "single family": "SingleFamilyResidence", land: "UnimprovedLand"
  };
  const typeKey = Object.keys(typeMap).find(k =>
query.toLowerCase().includes(k));

  let maxPrice = null;
  if (priceMatch) {
    maxPrice = Number(priceMatch[1].replace(/,/g, ""));
```

```
    if (priceMatch[2]?.toLowerCase() === "k") maxPrice *= 1000;
    if (priceMatch[2]?.toLowerCase() === "m") maxPrice *= 1_000_000;
  }

  return {
    city:     cityMatch?.[1]?.trim() || null,
    maxPrice,
    beds:     bedsMatch  ? Number(bedsMatch[1])  : null,
    baths:    bathsMatch ? Number(bathsMatch[1]) : null,
    sqft:     sqftMatch  ? Number(sqftMatch[1])  : null,
    type:     typeKey ? typeMap[typeKey] : null,
    pool:     poolMatch ? "True" : null,
    hasView:  viewMatch  ? "True" : null,
  };
}
```

Deliverable

An OpenClaw skill that accepts a free-text query and returns a structured filter object, validated against at least 10 test queries.

WEEK 3 1 Week

MLS Database Integration

Goal

Connect OpenClaw to both MLS tables with a robust, parameterized query layer. Focus on safe SQL construction, efficient pagination, and clean result formatting for downstream agents.

MySQL Connection Module

```
import mysql from "mysql2/promise";

const pool = mysql.createPool({
  host:     process.env.MYSQL_HOST,
  user:     process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE,
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
});

export async function query<T>(sql: string, params: any[] = []): Promise<T[]> {
  const [rows] = await pool.execute(sql, params);
  return rows as T[];
}
```

Active Listing Search (rets_property)

```
export async function searchActiveListings(filters: PropertyFilters, page = 1,
limit = 10) {
  const offset = (page - 1) * limit;
  let sql = `
    SELECT
      L_ListingID, L_DisplayId, L_Address, L_City, L_Zip,
      L_SystemPrice AS price, L_Keyword2 AS beds, LM_Dec_3 AS baths,
      LM_Int2_3 AS sqft, L_Type_ AS type, L_Status AS status,
      LMD_MP_Latitude AS lat, LMD_MP_Longitude AS lng,
      YearBuilt, AssociationFee, DaysOnMarket,
      PoolPrivateYN, ViewYN, FireplaceYN, PhotoCount,
      LA1_UserFirstName, LA1_UserLastName, LO1_OrganizationName
    FROM rets_property WHERE L_Status = "Active"
  `;
  const params: any[] = [];

  if (filters.city)     { sql += " AND L_City = ?";
params.push(filters.city); }
  if (filters.maxPrice) { sql += " AND L_SystemPrice <= ?";
params.push(filters.maxPrice); }
  if (filters.beds)     { sql += " AND L_Keyword2 >= ?";
params.push(filters.beds); }
  if (filters.baths)    { sql += " AND LM_Dec_3 >= ?";
params.push(filters.baths); }
```

```
  if (filters.sqft)     { sql += " AND LM_Int2_3 >= ?";
params.push(filters.sqft); }
  if (filters.type)     { sql += " AND L_Type_ = ?";
params.push(filters.type); }
  if (filters.pool)     { sql += " AND PoolPrivateYN = ?";
params.push(filters.pool); }
  if (filters.hasView)  { sql += " AND ViewYN = ?";
params.push(filters.hasView); }

  sql += " ORDER BY L_SystemPrice ASC LIMIT ? OFFSET ?";
  params.push(limit, offset);

  return query<ListingRow>(sql, params);
}
```

Sold Comps Query (california_sold)

```
export async function getSoldComps(city: string, months = 12) {
  const sql = `
    SELECT
      ListingKey, UnparsedAddress, City, CloseDate, ClosePrice,
      OriginalListPrice, ListPrice, DaysOnMarket,
      BedroomsTotal, BathroomsTotalInteger, LivingArea,
      PropertyType, PropertySubType, YearBuilt,
      ListAgentFullName, ListOfficeName, BuyerOfficeName
    FROM california_sold
    WHERE City = ?
      AND CloseDate >= DATE_SUB(CURDATE(), INTERVAL ? MONTH)
      AND PropertyType = "Residential"
    ORDER BY CloseDate DESC
    LIMIT 50
  `;
  return query<SoldRow>(sql, [city, months]);
}
```

Deliverable

Fully functional OpenClaw skill: accepts NLP filters from Week 2, queries rets_property and/or california_sold, returns formatted property cards.

WEEK 4 1 Week

Conversational Property Search Agent

Goal

Transform the single-turn query tool from Week 3 into a multi-turn conversational experience. The agent asks follow-up questions, remembers preferences within a session, and refines results iteratively.

Example Conversation

User: "Find homes in Irvine" Agent: "What is your budget?" User: "Under $1.2M" Agent: "Any preferences — condo, townhome, or single family?" User: "Single family with at least 3 beds" Agent: [returns filtered results from rets_property]

Session Memory Implementation

```
interface UserSession {
  city?: string;
  maxPrice?: number;
  beds?: number;
  baths?: number;
  type?: string;
  pool?: string;
  lastResults?: ListingRow[];
  conversationStep: number;
}

const sessions = new Map<string, UserSession>();

export function getSession(userId: string): UserSession {
  if (!sessions.has(userId)) {
    sessions.set(userId, { conversationStep: 0 });
  }
  return sessions.get(userId)!;
}

export function updateSession(userId: string, updates: Partial<UserSession>) {
  const session = getSession(userId);
  sessions.set(userId, { ...session, ...updates });
}

export function clearSession(userId: string) {
  sessions.delete(userId);
}
```

Deliverable

Multi-turn WhatsApp conversation that progressively refines a search query and returns rets_property results with address, price, beds/baths, and photo count.

WEEK 5 1 Week

Market Statistics Agent

Goal

Build a market analytics engine powered by california_sold — the 439K-record historical comps table. Agents can answer questions like "Is now a good time to buy in San Diego?" or "What is the average price per sq ft in Pasadena?"

Core Market Metrics

- Average and median close price by city, ZIP, or property type
- Price per square foot trends over time
- List-to-close price ratio (negotiation leverage indicator)
- Average days on market by city and month
- Inventory comparison: rets_property active count vs. california_sold volume
- Month-over-month and year-over-year trend comparisons

SQL: City Market Summary

```
SELECT
  City,
  COUNT(*)                                        AS sold_count,
  ROUND(AVG(ClosePrice), 0)                       AS avg_close_price,
  ROUND(AVG(ClosePrice / NULLIF(LivingArea,0)),0) AS avg_price_per_sqft,
  ROUND(AVG(DaysOnMarket), 1)                     AS avg_dom,
  ROUND(AVG(ClosePrice / NULLIF(ListPrice,0)) * 100, 1) AS list_to_close_pct
FROM california_sold
WHERE PropertyType = 'Residential'
  AND CloseDate >= DATE_SUB(CURDATE(), INTERVAL 12 MONTH)
  AND LivingArea > 0
GROUP BY City
ORDER BY sold_count DESC
LIMIT 25;
```

Python Trend Analysis

```
import pandas as pd
import mysql.connector
from sqlalchemy import create_engine

engine = create_engine(
  "mysql+mysqlconnector://idx_user:password@localhost/idx_exchange"
)

# Monthly price trends for a city
def get_price_trend(city: str, months: int = 24):
    query = """
        SELECT
```

```
            DATE_FORMAT(CloseDate, "%Y-%m") AS month,
            COUNT(*)                        AS sales,
            ROUND(AVG(ClosePrice), 0)       AS avg_price,
            ROUND(AVG(DaysOnMarket), 1)     AS avg_dom
        FROM california_sold
        WHERE City = %s
          AND PropertyType = "Residential"
          AND CloseDate >= DATE_SUB(CURDATE(), INTERVAL %s MONTH)
        GROUP BY DATE_FORMAT(CloseDate, "%Y-%m")
        ORDER BY month
    """
    df = pd.read_sql(query, engine, params=[city, months])
    df["price_change_pct"] = df["avg_price"].pct_change() * 100
    return df
```

Deliverable

OpenClaw skill that responds to market questions with data-backed summaries: median price, DOM, list-to-close ratio, and 12-month trend for any California city.

WEEK 6
1 Week

Embeddings & Vector Search

Goal

Go beyond keyword filtering. Use OpenAI embeddings to find semantically similar properties — so a query like "charming craftsman with mountain views and character" matches relevant listings even without exact keyword overlap.

> **Why This Matters**
> rets_property.L_Remarks contains rich full-text listing descriptions (FULLTEXT indexed). Embedding these enables similarity search that structured SQL filters cannot achieve.

Embedding Generation

```
from openai import OpenAI
import numpy as np

client = OpenAI()

def get_embedding(text: str, model="text-embedding-3-small") -> list[float]:
    text = text.replace("\n", " ").strip()[:8000]  # max token safety
    response = client.embeddings.create(model=model, input=text)
    return response.data[0].embedding

# Build embedding for a listing (combine key fields)
def build_listing_embedding(row: dict) -> list[float]:
    text = f"""
        {row["L_Type_"]} in {row["L_City"]}, CA.
        {row["L_Keyword2"]} beds, {row["LM_Dec_3"]} baths.
        {row["LM_Int2_3"]} sq ft. Built {row["YearBuilt"]}.
        Price: ${row["L_SystemPrice"]:,}.
        {row.get("L_Remarks", "")}
    """.strip()
    return get_embedding(text)
```

Cosine Similarity Search

```
from sklearn.metrics.pairwise import cosine_similarity

def find_similar_listings(
    query: str,
    listing_embeddings: list[tuple[str, list[float]]],
    top_k: int = 5
) -> list[str]:
    """Return top_k listing IDs most similar to the query."""
    query_vec = np.array(get_embedding(query)).reshape(1, -1)
    scores = []
    for listing_id, emb in listing_embeddings:
        sim = cosine_similarity(query_vec, np.array(emb).reshape(1, -1))[0][0]
```

```
            scores.append((listing_id, float(sim)))
    scores.sort(key=lambda x: x[1], reverse=True)
    return [lid for lid, _ in scores[:top_k]]
```

Deliverable

Semantic property search skill that takes a free-text description and returns the top 5 most similar active listings from rets_property using embedding cosine similarity.

WEEK 7
1 Week

Recommendation Engine

Goal

Build a hybrid recommendation engine that combines structured similarity scoring with embedding similarity. Given a listing a user likes, surface comparable active listings from rets_property and validate recommendations against california_sold pricing data.

Hybrid Scoring Model

```
def calculate_similarity_score(
    target: dict,
    candidate: dict,
    target_emb: list[float],
    candidate_emb: list[float]
) -> float:
    score = 0.0

    # Structured similarity (60% of total score)
    price_diff = abs(target["L_SystemPrice"] - candidate["L_SystemPrice"])
    if price_diff < 50_000:   score += 20
    elif price_diff < 150_000: score += 12
    elif price_diff < 300_000: score += 5

    if target["L_Keyword2"] == candidate["L_Keyword2"]:       score += 15
    if target["L_City"]     == candidate["L_City"]:           score += 15

    sqft_diff = abs(target["LM_Int2_3"] - candidate["LM_Int2_3"])
    if sqft_diff < 300:  score += 10
    elif sqft_diff < 700: score += 5

    # Semantic similarity (40% of total score)
    from sklearn.metrics.pairwise import cosine_similarity
    import numpy as np
    sem_sim = cosine_similarity(
        np.array(target_emb).reshape(1,-1),
        np.array(candidate_emb).reshape(1,-1)
    )[0][0]
    score += sem_sim * 40

    return round(score, 2)
```

Comp Validation via california_sold

```
# Check if recommended price is supported by recent comps
def validate_with_comps(city: str, sqft: int, price: int) -> dict:
    sql = """
        SELECT
          AVG(ClosePrice / NULLIF(LivingArea,0)) AS avg_ppsf,
          COUNT(*) AS comp_count
        FROM california_sold
        WHERE City = %s AND PropertyType = 'Residential'
```

```
        AND LivingArea BETWEEN %s AND %s
        AND CloseDate >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
    """
    result = query(sql, [city, sqft * 0.8, sqft * 1.2])
    avg_ppsf = result[0]["avg_ppsf"] or 0
    comp_price = avg_ppsf * sqft
    return {
        "comp_price":  round(comp_price),
        "list_price":  price,
        "comp_count":  result[0]["comp_count"],
        "delta_pct":   round((price - comp_price) / comp_price * 100, 1)
    }
```

Deliverable

Recommendation skill that surfaces top 5 similar active listings with a comp-validated price assessment sourced from california_sold data.

WEEK 8
1 Week

Retrieval-Augmented Generation (RAG)

Goal

Build a document-aware RAG assistant that can answer questions about real estate concepts, MLS field definitions, and market terminology — grounding its answers in authoritative source documents rather than hallucinating.

Knowledge Sources

- **MLS field definitions (column mappings from both SQL tables)**
- IDX Exchange internal documentation
- Real estate terminology glossary (DOM, escrow, comps, cap rate, etc.)
- California real estate law summaries and disclosure requirements
- Market reports sourced via the market analytics agent (Week 5)

RAG Pipeline

```
# 1. Chunk documents
def chunk_text(text: str, chunk_size=600, overlap=100) -> list[str]:
    chunks, start = [], 0
    while start < len(text):
        end = min(start + chunk_size, len(text))
        chunks.append(text[start:end])
        start += chunk_size - overlap
    return chunks

# 2. Index chunks
def index_documents(docs: list[dict]) -> list[dict]:
    indexed = []
    for doc in docs:
        for chunk in chunk_text(doc["content"]):
            indexed.append({
                "source": doc["title"],
                "chunk":  chunk,
                "embedding": get_embedding(chunk)
            })
    return indexed

# 3. Retrieve relevant chunks for a query
def retrieve(query: str, index: list[dict], top_k=4) -> list[dict]:
    q_emb = np.array(get_embedding(query)).reshape(1,-1)
    scored = [
        (doc, cosine_similarity(q_emb, np.array(doc["embedding"]).reshape(1,-
1))[0][0])
        for doc in index
    ]
    scored.sort(key=lambda x: x[1], reverse=True)
    return [doc for doc, _ in scored[:top_k]]

# 4. Generate grounded answer
```

```
def rag_answer(query: str, index: list[dict]) -> str:
    chunks = retrieve(query, index)
    context = "\n\n".join(c["chunk"] for c in chunks)
    prompt = f"Answer using only the context below:\n\n{context}\n\nQuestion:
{query}"
    resp = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": prompt}]
    )
    return resp.choices[0].message.content
```

Deliverable

RAG skill that answers "What does DOM mean?", "What columns are in california_sold?", and "What is a list-to-close ratio?" using indexed source documents.

WEEK 9 1 Week

Multi-Agent Orchestration

Goal

Combine all specialized agents into a single intelligent coordinator. The orchestrator analyzes each incoming query and routes it — or splits it across multiple agents — to produce a comprehensive, unified response.

Agent Registry

- **propertySearchAgent — queries rets_property with structured filters**
- marketStatsAgent — aggregates california_sold for trends and comps
- recommendationAgent — surfaces similar listings with comp validation
- ragAgent — answers conceptual and definitional questions
- emailDraftAgent — composes formatted property or market summaries

Orchestrator Logic

```
export async function orchestrate(query: string, userId: string) {
  const intent = await classifyIntent(query);
  // intent: "search" | "market" | "recommend" | "knowledge" | "mixed"

  switch (intent) {
    case "search":
      return await propertySearchAgent(query, userId);

    case "market":
      return await marketStatsAgent(query);

    case "recommend":
      const session = getSession(userId);
      return await recommendationAgent(session.lastResults?.[0]);

    case "knowledge":
      return await ragAgent(query);

    case "mixed": {
      const [listings, stats] = await Promise.all([
        propertySearchAgent(query, userId),
        marketStatsAgent(query)
      ]);
      return formatCombinedResponse(listings, stats);
    }

    default:
      return { response: "I'm not sure how to help with that. Try asking about
properties or market trends." };
  }
}
```

Example Query

"Find me affordable homes in Pasadena and tell me whether prices are rising." → orchestrator routes to propertySearchAgent + marketStatsAgent in parallel, merges results.

Deliverable

A single OpenClaw entry point that correctly routes queries across all five agents, with a test suite demonstrating mixed-intent handling.

WEEK 10
1 Week

WhatsApp Communication Layer

Goal

Wire the complete orchestrator to WhatsApp as the primary conversational interface. Users interact entirely through WhatsApp — searching for homes, asking market questions, and receiving formatted property summaries — all in real time.

Architecture

WhatsApp → OpenClaw Channel → orchestrate() → [agents] → rets_property / california_sold → formatted response → WhatsApp

Message Handler

```
export async function onWhatsAppMessage(message: string, userId: string) {
  // Typing indicator while processing
  await sendTypingIndicator(userId);

  try {
    const result = await orchestrate(message, userId);
    return formatForWhatsApp(result);
  } catch (err) {
    console.error("Orchestration error:", err);
    return "Sorry, I hit an issue. Please try again.";
  }
}

function formatForWhatsApp(result: AgentResult): string {
  if (result.listings) {
    return result.listings.slice(0,5).map(l =>
      `🏠 *${l.L_Address}, ${l.L_City}*\n` +
      `    💰 $${l.price.toLocaleString()} | 🛏 ${l.beds}bd/${l.baths}ba | 📐 ${l.sqft} sqft\n` +
      `    📅 ${l.DaysOnMarket} days on market`
    ).join("\n\n");
  }
  return result.response || "No results found.";
}
```

Deliverable

End-to-end WhatsApp assistant handling property search, market questions, and recommendations with clean formatted responses.

WEEK 11 1 Week

Email Agents & Safety Guardrails

Goal

Add automated email workflows for listing alerts, market reports, and property summaries — with strict human-approval guardrails so no email is ever sent autonomously without explicit confirmation.

Email Use Cases

- New listing alerts matching a saved user search (from rets_property)
- Weekly market report compiled from california_sold analytics
- Property summary card with address, photos, price, comps
- Personalized recommendation digest

Email Tool (with Approval Gate)

```
import nodemailer from "nodemailer";

const transporter = nodemailer.createTransport({
  service: "gmail",
  auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASSWORD },
});

// STEP 1: Draft — never send without approval
export async function draftEmail(to: string, subject: string, body: string) {
  return { draft: { to, subject, body }, status: "pending_approval" };
}

// STEP 2: Send only after explicit human confirmation
export async function sendApprovedEmail(draft: EmailDraft): Promise<void> {
  await transporter.sendMail({
    from: process.env.EMAIL_USER,
    to: draft.to,
    subject: draft.subject,
    html: draft.body,
  });
}
```

Safety Rules — Non-Negotiable

NEVER Do This	ALWAYS Do This
Send emails without explicit user approval	Queue as draft, show preview, require confirmation
Expose API keys or credentials in logs	Store secrets in .env only, never log them

Export or bulk-download full MLS datasets	Return result sets of ≤50 rows per query
Operate autonomously without human oversight	Every destructive/outbound action requires approval

Deliverable

Email agent with draft-then-approve workflow. Weekly market report template populated from california_sold data. Safety guardrail test suite passing.

WEEK 12 1 Week

Capstone Demo — Final Project

Final Project: IDX Multi-Agent Real Estate Assistant

Your capstone integrates every component built across the program into a single, production-ready multi-agent AI assistant. It must be fully operational, documented, and demo-ready.

Required Features

Feature	Source Database(s)
Natural language property search	rets_property
Conversational multi-turn memory	rets_property
Market analytics & trends	california_sold
Comp-validated price assessments	california_sold
Semantic similarity search	rets_property (L_Remarks embeddings)
Recommendation engine	rets_property + california_sold
RAG knowledge assistant	Indexed docs + MLS field definitions
Multi-agent orchestration	Both tables
WhatsApp communication layer	Both tables
Email drafting with approval gate	Both tables

Deliverables Checklist

- GitHub repository with clean commit history and README
- Architecture diagram (full multi-agent flow with both databases)
- Schema annotation document — your field-usage notes
- 15-minute live demo via WhatsApp + screen share
- Demo video recording (backup)
- Written reflection: what worked, what you would change

Resume Outcome

Copy-ready resume bullet:

Built a production multi-agent AI assistant using OpenClaw, integrating natural language search over 667K+ MLS records (rets_property + california_sold), conversational memory, OpenAI vector embeddings, semantic similarity search, comp-validated recommendations, retrieval-augmented generation (RAG), WhatsApp communication, and email workflow automation with human-in-the-loop safety guardrails.